

目录

1 需求分析	2
1.1 项目背景	2
1.2 用户类及其特征	2
1.3 产品功能	2
2 整体设计	4
2.1 体系结构设计	4
2.2 分层结构	5
2.2.1 表现层	5
2.2.2 业务层	7
2.2.3 持久层	8
2.2.4 数据库层	8
2.3 数据处理流程	9
3 设计模式	10
3.1 概述	10
3.2 设计模式应用	11
3.2.1 创建型模式	11
3.2.2 结构型模式	11
3.2.3 行为型模式	12
4 测试	14
4.1 概述	14
4.2 测试结果	15
4.2.1 内容测试	15
4.2.2 数据库测试	16
4.2.3 用户接口测试	16
4.2.4 导航测试	17
4.2.5 配置测试	17
4.2.6 可用性测试	18
4.2.7 边界测试	18
4.2.8 兼容性测试	18
4.2.9 性能测试	19
4.3 总结分析	19

1 需求分析

1.1 项目背景

该项目开发的软件为一个辟谣平台及谣言收集系统。2020年初，中国爆发大规模新型冠状病毒疫情，导致社会整体停工大约两个月。在此期间，由于传播时的错误及部分人员的夸大化描述下，诞生了许多奇怪的谣言，大部分谣言真假难辨，对广大群众的日常生活造成了巨大的影响。鉴于网络技术的普及，线上传播逐渐成为谣言生成的主要途径。

信息化的谣言收集及澄清系统能够帮助政府更好的预防谣言的传播，更方便的将一些晦涩的科学知识传授给广大群众。与传统的辟谣方式相比，集中化的辟谣平台效率更高，权威性更强，传播度更广，并且拥有主动收集谣言信息，主动发布真实内容，实时权威解读等多种传统辟谣方式所不具备的功能。另一方面，群众能够通过浏览辟谣平台，预防谣言传播，学习相关的科学知识，了解相关的疫情动态，从而在疫情防治的过程中，保持理性的头脑，帮助政府共克时艰。

互联网时代下，各种信息眼花缭乱，百姓很难在各种消息之间保持正确的观点，所以我们希望能够通过这样一个辟谣平台，实现集实时收集谣言信息，权威认证相关资讯，官方发布正式消息，实时疫情动态分析等功能的现代化辟谣平台，帮助相关部门宣传正确的疫情相关知识，避免因谣言而造成的群众恐慌或信息闭塞。

1.2 用户类及其特征

用户类型	功能特征
管理员	系统最高权限操作者，能够对现有的谣言数据进行标记修改及删除或发布
辟谣者	部分经过认证的相关专家或政府工作人员，拥有对未定谣言的标记发布权，能够对用户发出的疑问进行回答和整合发布，可以直接发布相关谣言澄清信息
普通用户	网站的主要适用群体，能够在页面上浏览相关信息，获取最新的谣言情报和疫情动态，能够对辟谣者发起咨询（信息将进入待定谣言库）

1.3 产品功能

产品使用者可分为上述的三种用户，依照各个用户所拥有的权限，我们的辟谣平台的功能集中于以下几个方面：

功能类型	适用群体
登录	管理员、普通用户、辟谣者
注销	管理员、普通用户、辟谣者
查询相关信息	管理员、普通用户、辟谣者
查看高关注度信息	管理员、普通用户、辟谣者
发起相关咨询	管理员、普通用户、辟谣者
发布澄清信息	管理员、辟谣者
编辑谣言信息	管理员、辟谣者
发布公告	管理员
更改用户权限	管理员

具体的功能描述：

登录。用户在使用系统前必须进行登录。登录时可以选择管理员、辟谣者、普通用户三个身份。用户登录系统根据用户输入的账户与密码验证其身份，具体的验证功能由登录系统提供。验证身份成功后，用户方可进入该系统进行后续操作。

注销。用户在登录之后的系统页面内可以随时选择注销，该操作让用户退出登录状态。

查询相关信息。用户在登录之后的系统主页面上方可见一格搜索框，单击进入搜索页面。可透过单关键词搜索及多关键词搜索，针对已被审核后发出的辟谣信息内容进行检查并条列显示，再单击进入查看内容。

查看高关注度信息。用户在登录之后的系统主页面搜索框之下，可见条列式的信息推送，并提供用户不同排列方式选择：按阅读量排列，即为热搜模式；按更新时间排列，即为最新消息模式。

发起相关咨询。用户在登录之后的系统主页面可见一“提问较真”按钮，单击进入提问页面，透过文字输入（是否扩展支持图片再议，此处亦可对信息做分类标注）对辟谣者发起咨询，该部分上传信息将进入待定谣言库。

发布澄清消息。管理员可以针对某流传的信息发布澄清消息，提交文字以及多媒体材料佐证，对消息进行证实或证伪。澄清信息中包括原未经验证的信息，用以作证的材料以及辟谣或证实标签，所有权限组的用户都可看到已辟谣或证实的信息。

编辑澄清信息。中权限组可以对已发布的澄清信息进行编辑，编辑过程中可以对已有的材料进行修改、删除或添加新的材料。随着事件的跟进，相关信息的真实与否可能会产生变化，通过编辑功能使澄清信息更加完善。并且编辑前的版本信息可见，查看澄清信息的用户可以看到事件的全貌，更具有说服力。

发布公告。管理员可以发布公告，公告将在界面置顶，所有权限组都可以看到公告，且公告不可修改。通过此功能，高权限组可以发布与平台相关的信息，或是针对某条流传度广、造成误解程度深的信息进行辟谣，置顶位置的公告可以让所有用户第一时间看到，消除不实信息带来的影响。

编辑澄清信息。对于特殊情况下，需要及时修改的信息，高权限组的管理员用户可以直接对单条谣言信息进行编辑，编辑后将覆盖原有的谣言信息。

更改用户权限。管理员可以提升或降低单一用户权限组别，更方便、快捷地认证辟谣者，或是撤销权限。

2 整体设计

2.1 体系结构设计

软件结构风格是描述某一特定应用领域中系统组织方式的惯用模式。体系结构风格定义了一个系统家族，即一个体系结构定义一个词汇表和一组约束。词汇表中包含一些构件和连接件组合起来的。体系结构风格反映了领域中众多系统所共有的结构和语义特性，并指导如何将各个模块和子系统有效地组织成一个完整的系统。按这种方式理解，软件体系结构风格定义了用于描述系统的术语表和一组指导构件系统的规则。在过去数百万的计算机系统中，绝大多数可以归结成以下少数模式中的一种：

- 以数据为中心的体系结构（Data-centered architectures）
- 数据流体系结构（Data flow architectures）
- 调用和返回体系结构（Call and return architectures）
- 面向对象体系结构（Object-oriented architectures）
- 分层体系结构（Layered architectures）

本次对辟谣系统的设计使用了分层体系结构。通过将系统分层非常有利于开发，充分体现了“高内聚低耦合”的思想，因此分层架构是最常见的体系结构，被架构师、设计师和开发者所熟知。分层体系结构和大多数公司传统的IT以及组织结构非常接近，使得它成为大多数业务应用开发任务的天然选择。

分层体系结构中的每一层都在应用中扮演特定的角色。比如说，显示层会负责处理所有的用户接口和交互逻辑，而一个业务层主要负责执行特定的业务和用户请求密切联系的任务。架构中的每一层都是满足某种业务请求需要做的事情的一个抽象。显示层不需要知道或者关心怎么获取用户数据；它只需要以某种格式在屏幕上显示信息。类似地，业务层也不需要关心数据是以什么格式显示的甚至数据时从哪来的；它只需要从持久层拿到数据，对数据按照业务逻辑进行处理，然后将信息传给显示层。

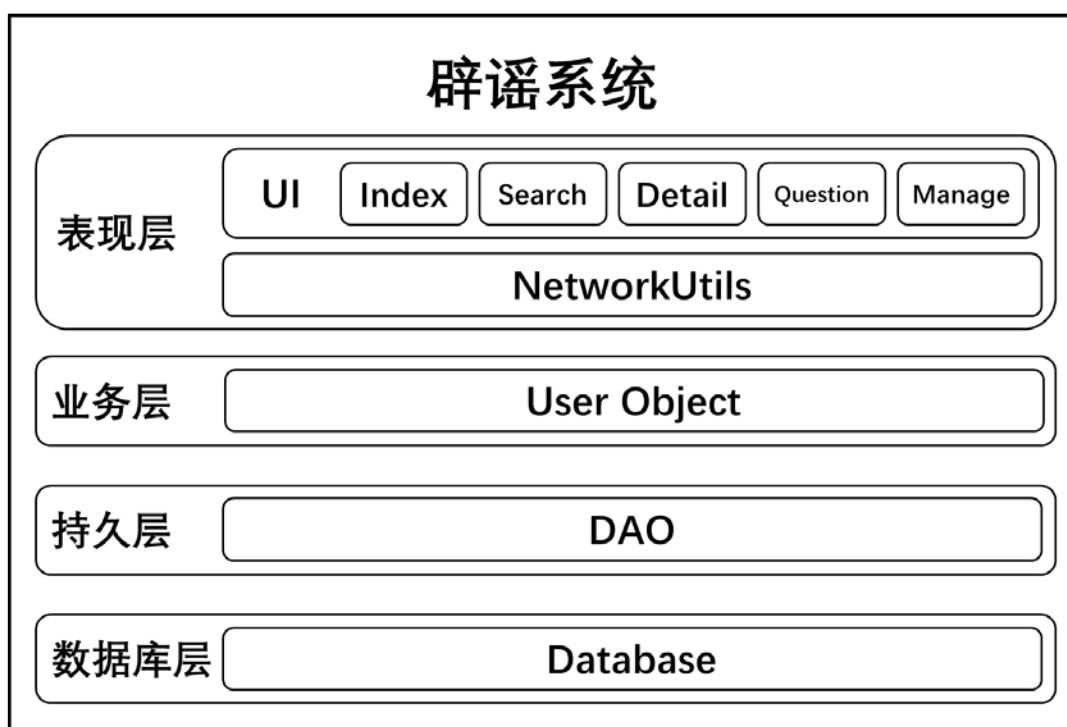
2.2 分层结构

在前一节中，我们定义了软件所使用的体系结构风格，我们认为使用分层体系结构可以使软件结构更加有条理，软件开发更加高效，因此此处我们还需定义软件的层次结构。根据软件的逻辑结构以及功能，将系统分为四个层次：

- 表现层（Presentation Layer）：提供易于交互的用户界面
- 业务层（Business Layer）：处理用户请求，包装业务逻辑
- 持久层（Persistence Layer）：提供封装好的数据库操作
- 数据库层（Database Layer）：构建数据库

层级结构中层与层相互隔离。层的隔离意味着在某一层的改变通常不会影响到其他层的组件。如果我们允许显示层直接调用持久层，那么SQL的改变将会对业务层和显示层都造成影响，因此导致一个非常紧耦合的应用，各个组件会有非常多的相互依赖。这种架构就会变得非常难改变。层的隔离的概念也意味着每一层与其他层都是独立的，因此也不需要知道架构中其他层的工作。

分层体系结构图如下：



2.2.1 表现层

表现层位于分层构架的最上层，与用户直接接触。表现层的主要功能是实现系统数据的传入与输出，在此过程中不需要借助逻辑判断操作就可以将数据传送到系统中进行处理，处理后会将会处理结果反馈到表现层中。换句话说，表现层就是实现用户界面功能，将

用户的需求传达和反馈，保证用户体验。

该层主要分为两大模块：用户界面模块（User Interface）和网络工具模块（NetworkUtils）。

用户界面模块主要使用的是web前端的React框架，用户（普通用户，管理员，专家）将通过这一层与内部数据进行交互，普通用户可以浏览最新的辟谣信息，专家可以对谣言进行澄清，对未定的谣言进行相关的调查后进行发布，管理员可以对现有的错误证言进行修改，从而防止出现特殊情况。

在进行 UI 设计时，需要充分考虑用户的需求，需要将以下原则融入 UI 的设计中：

- 设计简洁明了，考虑用户习惯

本产品主要面向的是移动端用户，移动端的前端开发主要要求UI尽可能的简单，要在大部分相对较小的屏幕上合理安排所有的功能。用简单的页面逻辑去简化用户的操作复杂度，从而保证用户的使用体验以及用户留存度。

- 美观大方

系统界面的设计在视觉效果上便于理解和使用。同时，也可以让界面更为美观，让用户在使用感觉赏心悦目，提高用户的使用体验。

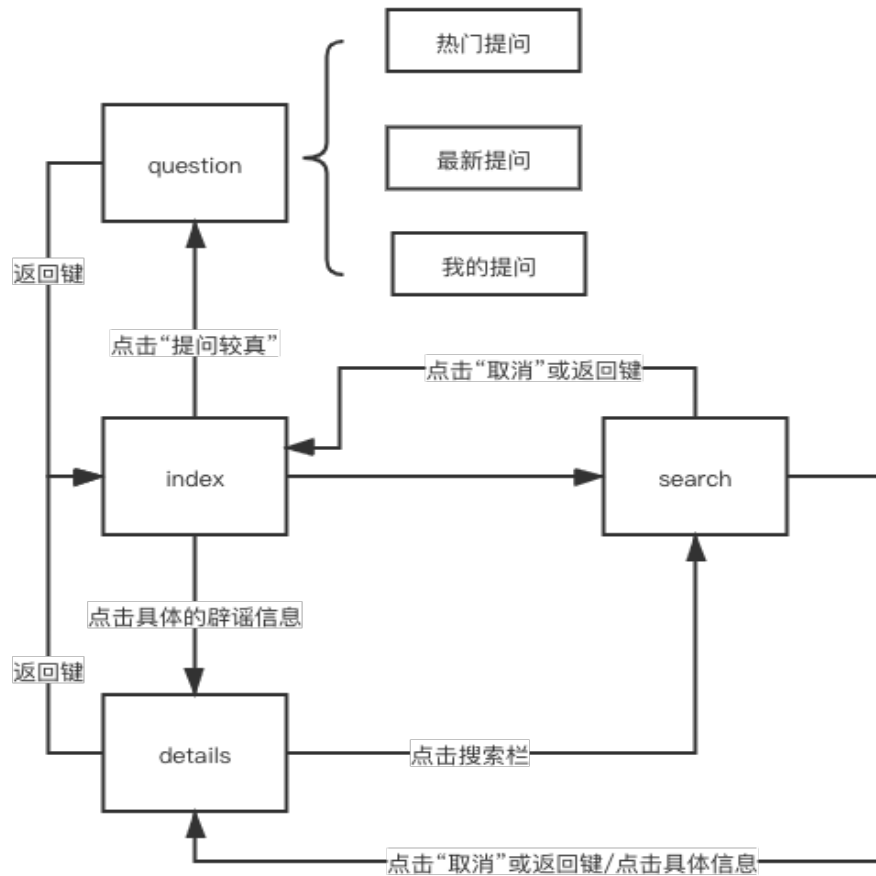
- 一致性

界面的结构必须清晰且一致，各页面之间要求大体UI设计相近，从而保证界面风格的一致。

用户界面主要负责的是将数据展示给使用者，以及将用户的输入传达到业务层，从而对数据进行修改。所以在这一层，我们所负责的对象主要有三者：普通用户、查证者、管理员。而功能模块主要可以分为：首页模块（Index），详情页面（Detail），查询功能（Search），问题咨询（Question），管理页面（Manage）。他们都依赖于UI模块和网络模块（NetworkUtils）。

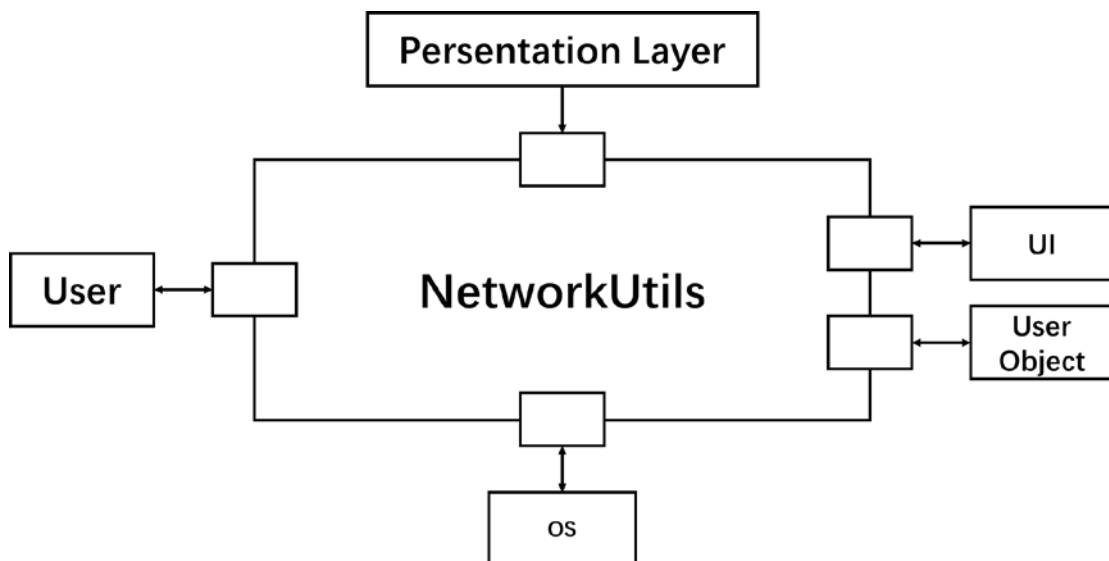
首页即展示最新的辟谣公示的地方，可以引导到详情页面、搜索页面、问题咨询页面；详情页面为一个辟谣信息的具体内容展示，主要负责向后端请求对应编号的辟谣信息的具体内容，然后在页面上渲染显示；搜索页主要负责对用户的输入向后端发起请求，获取热搜、搜索结果，可以引导到上一页面和具体详情；咨询页是一个简单的论坛机制，用户可以在这里快速发起咨询，并且可以查看最新的咨询问题和热门咨询。每个咨询可以得到回复，用户也可以查看自己的历史咨询内容；管理页负责对查证者和管理员的操作进行可视化展示。查证者可以从数据库中查看人们的咨询，并且发布最新的辟谣信息或回复用户的咨询。而管理员可以修改已发布的信息，以及查证者相关权限的功能。

交互设计是UI层的核心之一，我们要保证各功能的页面交互以及前后端的交互流畅。用户进入webapp后的流程图（入口为首页index）为：



查证者和管理员的管理界面因为功能较多，我们将选择在电脑端单独进行管理。

网络工具模块主要负责集成各页面的查询功能，负责统一发送和解析各页面的请求信息。基于fetch接口包装，从而严格控制请求的规范。其体系结构环境图如下：



2.2.2 业务层

业务层的关注点主要集中在业务规则的制定、业务流程的实现等与业务需求有关的系统设计，其位置处于持久层与表现层中间，起到了数据交换中承上启下的作用。一个理想的分层式架构，应该是支持可抽取、可替换的，用以最大化的发挥其位于重要枢纽，指挥软件运作的价值。

业务层用User object与表现层透过NetworkUtil进行交互。并在Information Object调用到持久层的函数拿到数据后将之发送给表现层。设计前应与表现层和持久层确定好接口，以方便后续的整合。

2.2.3 持久层

持久层是在系统逻辑层面上，专着于实现数据持久化的一个相对独立的领域（Domain）。持久层是负责向（或者从）一个或者多个数据存储器中存储（或者获取）数据的一组类和组件。这个层必须包括一个业务领域实体的模型（即使只是一个元数据模型）。

具体来说，在node.js环境下express框架下引入mysql对象并实例化，配置mysql配置文件并使用createPool方法创建数据库的连接池（更有效地利用资源，减少连接所需的时间），在不同的操作中选择特定的sql语句进行数据获取或更改，控制数据库同时获取数据返送回业务层。

持久层主要起Model的作用，处理数据库并提供给控制器需要的数据，一般来说，需要对数据库开关进行控制并实现进行增删查改四大操作，因此我们将该层操作分成两大类：

- Connect/close

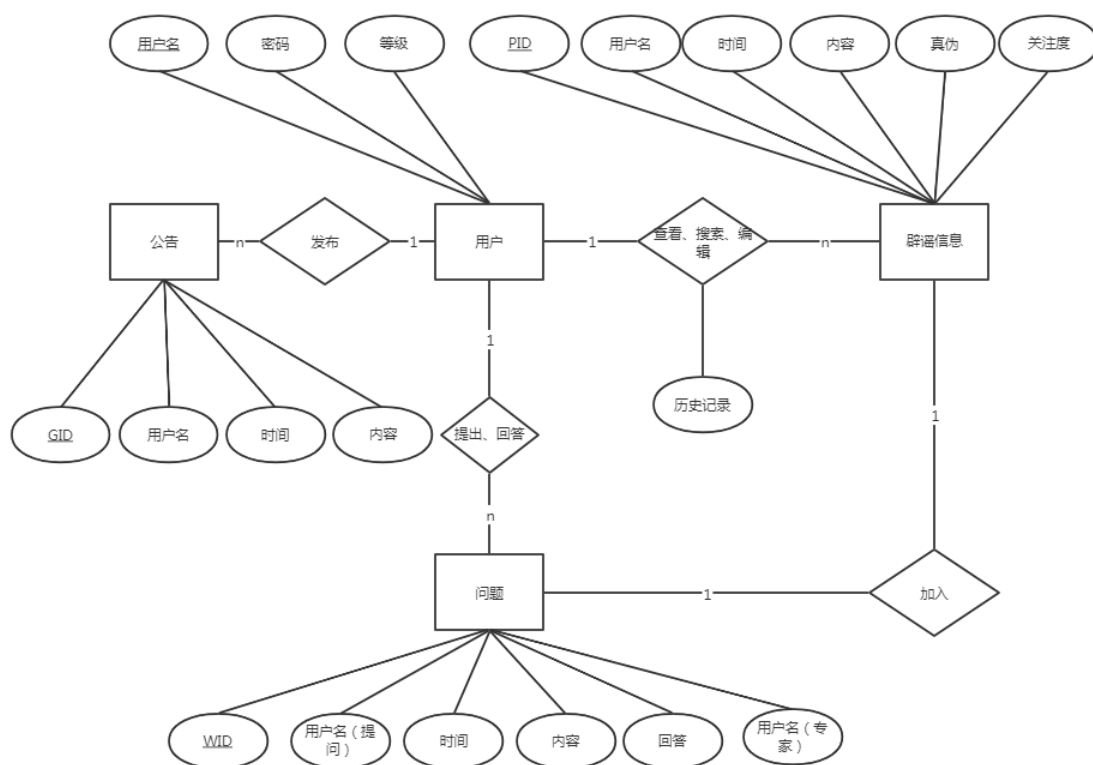
处理数据库连接和断开的请求，在持久层中配置数据库信息并采用连接池的方式处理连接请求，可以更好地利用资源，支持高并发操作。

- Insert/select/update/delete

增删查改操作从接口参数中获取sql语句的信息，将其拼接为相应处理的sql语句，对数据库中对对应表进行操作，同时将结果送回业务层。

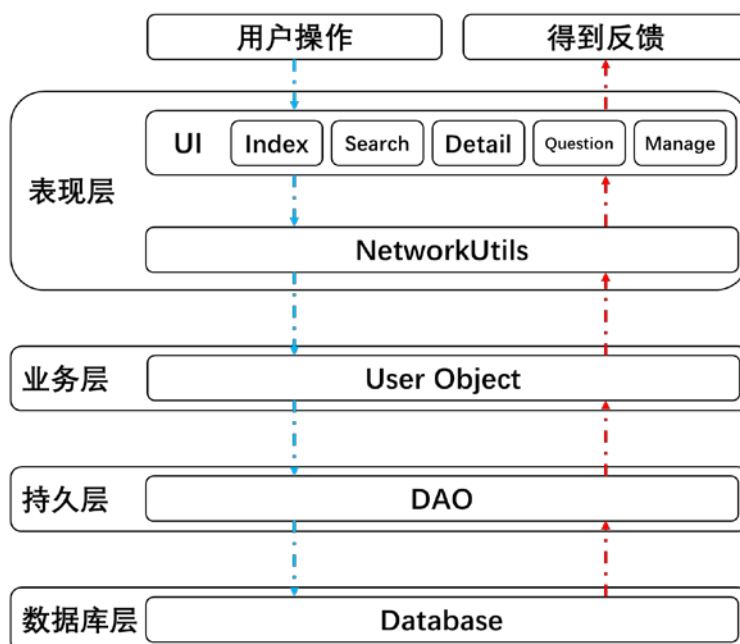
2.2.4 数据库层

数据库层中需要建立Web应用的数据库，既要保证稳定，又要充分支持应用的所有功能。ER图如下：

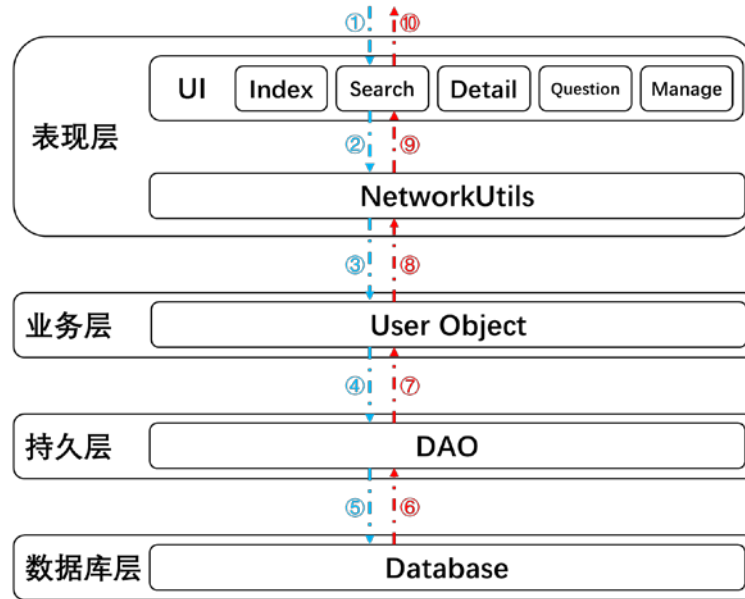


2.3 数据处理流程

使用分层设计构架可以清晰地看到数据的处理流程。用户在表现层产生请求，UI组件使用NetworkUtils向业务层发送请求；业务层解析请求，并处理成DAO的调用；持久层使用SQL语句访问数据库，数据库返回相应信息到持久层；之后，信息不断向上传递，直到表现层渲染页面，用户收到反馈。



以用户搜索辟谣信息为例，数据处理需要经过以下步骤：



包括：

- ① 用户点击进入搜索页面，输入关键词，点击搜索按钮；
- ② 表现层查询模块（Search）调用NetworkUtils中的函数，将关键词传递给NetworkUtils；
- ③ NetworkUtils向业务层发送查询请求；
- ④ 业务层User Object调用持久层DAO进行数据查询；
- ⑤ 持久层DAO用SQL语句在数据库中进行查询；
- ⑥ 数据库向持久层DAO返回查询结果，；
- ⑦ 持久层DAO向业务层User Object返回查询结果；
- ⑧ 业务层User Object向表现层NetworkUtils发送查询结果；
- ⑨ 表现层NetworkUtils将查询结果返回给查询模块（Search）；
- ⑩ 表现层根据查询结果渲染页面。

3 设计模式

3.1 概述

软件设计模式（Software Design Pattern），又称设计模式，是一套被反复使用、多数人知晓的、经过分类编目的、代码设计经验的总结。它描述了在软件设计过程中的一些不断重复发生的问题，以及该问题的解决方案。也就是说，它是解决特定问题的一系列套路，是前辈们的代码设计经验的总结，具有一定的普遍性，可以反复使用。其目的是为了提高代码的可重用性、代码的可读性和代码的可靠性。

设计模式是软件开发人员在软件开发过程中面临的一般问题的解决方案。这些解决方案是众多软件开发人员经过相当长的一段时间的试验和错误总结出来的。它可以使得设计者在设计时能够避免一些将会引起问题的紧耦合，增强软件设计的适应变化的能力。在软件设计中，合适的设计模式可以帮助软件设计者更好地进行软件设计，使得设计变得规范。

在《设计模式：可复用面向对象软件的基础》一书中，提到了23种设计模式，这是设计模式领域里程碑的事件，导致了软件设计模式的突破。这4位作者在软件开发领域里也以他们的“四人组”（Gang of Four, GoF）匿名著称，这23种设计模式统称为GoF设计模式，书中把它们分为创建型、结构型与行为型三种模式大类。

在这23种经典设计模式之外，一些模式的组合也广为人知（如MVC模式），我们也对这类设计模式进行了调查分析。

3.2 设计模式应用

3.2.1 创建型模式

3.2.1.1 单例（Singleton）模式

单例（Singleton）模式：一个类只有一个实例，且该类能自行创建这个实例的一种模式。单例模式是设计模式中最简单的模式之一。通常，普通类的构造函数是公有的，外部类可以通过“new构造函数()”来生成多个实例。但是，如果将类的构造函数设为私有的，外部类就无法调用该构造函数，也就无法生成多个实例。这时该类自身必须定义一个静态私有实例，并向外提供一个静态的公有函数用于创建或获取该静态私有实例。

单例模式在我们的项目中的一个应用在持久层，DAOFactory是一个生产DAO的工厂，工厂只需要有一个，因此调用Factory类的getInstance来返回这个唯一的实例。

3.2.1.2 工厂方法（Factory Method）模式

工厂方法（FactoryMethod）模式：定义一个创建产品对象的工厂接口，将产品对象的实际创建工作推迟到具体子工厂类当中。这满足创建型模式中所要求的“创建与使用相分离”的特点。我们把被创建的对象称为“产品”，把创建产品的对象称为“工厂”。

在前面提到的持久层的DAOFactory中，我们使用了工厂方法模式。DAOFactory为每个进行访问的用户生产DAO，从而达到创建对象的目的。

3.2.2 结构型模式

3.2.2.1 组合（Composite）模式

组合（Composite）模式：有时又叫作部分-整体模式，它是一种将对象组合成树状的层次结构的模式，用来表示“部分-整体”的关系，使用户对单个对象和组合对象具有一致的访问性。

组合模式在我们项目中的应用是前端的组件。我们的前端使用React进行开发，在前端的组件中有大量的组合、复用。

3.2.2.2 代理（Proxy）模式

代理（Proxy）模式：由于某些原因需要给某对象提供一个代理以控制对该对象的访问。这时，访问对象不适合或者不能直接引用目标对象，代理对象作为访问对象和目标对象之间的中介。

在我们的项目中，也使用了代理模式。表现层的NetworkUtils模块就是使用了这一模式。表现层的UI不便直接访问网络（会使代码混乱），因此使用NetworkUtils集成网络请求功能，这样就方便了对服务器的访问。

3.2.2.3 外观（Facade）模式

外观（Facade）模式：一种通过为多个复杂的子系统提供一个一致的接口，而使这些子系统更加容易被访问的模式。该模式对外有一个统一接口，外部应用程序不用关心内部子系统的具体的细节，这样会大大降低应用程序的复杂度，提高了程序的可维护性。

在分层体系结构中，外观模式尤为突出，层与层之间都是外观模式的应用。每一层（表现层除外）都为上一层提供了一系列接口。

3.2.2.4 适配器（Adaptor）模式

适配器（Adapter）模式：将一个类的接口转换成客户希望的另外一个接口，使得原本由于接口不兼容而不能一起工作的那些类能一起工作。适配器模式分为类结构型模式和对象结构型模式两种，前者类之间的耦合度比后者高，且要求程序员了解现有组件库中的相关组件的内部结构，所以应用相对较少些。

由于本次大程作业要求多人合作完成，因此难免出现类与类之间无法良好协作的情况。这是就需要应用适配器模式让类能协同工作。

3.2.3 行为型模式

3.2.3.1 观察者（Observer）模式

观察者（Observer）模式：指多个对象间存在一对多的依赖关系，当一个对象的状态发生改变时，所有依赖于它的对象都得到通知并被自动更新。这种模式有时又称作发布-订阅模式、模型-视图模式，它是对象行为型模式。

观察者模式在我们项目的前端部分应用很广泛，包括原生Js的事件监听、React的跨组

件通信等，都体现了观察者模式。

3.2.3.2 命令（Command）模式

命令（Command）模式的定义如下：将一个请求封装为一个对象，使发出请求的责任和执行请求的责任分割开。这样两者之间通过命令对象进行沟通，这样方便将命令对象进行储存、传递、调用、增加与管理。

在我们的项目中，NetworkUtils和业务层之间就通过命令模式进行交互。NetworkUtils通过网络发送请求，业务层接收到后将其解析，并根据解析结果选择不同的操作。其结构图如下：

3.2.3.3 备忘录（Memento）模式

备忘录（Memento）模式的定义：在不破坏封装性的前提下，捕获一个对象的内部状态，并在该对象之外保存这个状态，以便以后当需要时能将该对象恢复到原先保存的状态。该模式又叫快照模式。

备忘录模式是我们项目中管理模块的重要模式。使用备忘录模式可以轻松实现操作的撤销，这样对管理员非常方便。其结构图如下：

3.2.3.4 策略（Strategy）模式

策略（Strategy）模式：该模式定义了一系列算法，并将每个算法封装起来，使它们可以相互替换，且算法的变化不会影响使用算法的客户。策略模式属于对象行为模式，它通过对算法进行封装，把使用算法的责任和算法的实现分割开来，并委派给不同的对象对这些算法进行管理。

在我们的项目中，策略模式主要在业务层中应用。在业务层的实现中有可能用到一些算法，比如搜索功能需要对用户输入进行分词，分词算法有很多种，可以将每种算法封装起来，选择效果最好的进行应用。

3.2.3.5 迭代器（Iterator）模式

迭代器（Iterator）模式：提供一个对象来顺序访问聚合对象中的一系列数据，而不暴露聚合对象的内部表示。

我们的项目主题编程语言是JavaScript，JavaScript提供了丰富的迭代方法，如every、some、filter、map、forEach、reduce。这些迭代方法正是迭代器模式的体现。

以every为例，every用来查询数组中的每一项是否都满足条件：

```
function isEven(num) {
    return num % 2 == 0;
}
var nums = [1, 2, 3, 4, 5];
// var nums = [2, 4, 6, 8, 10]
var even = nums.every(isEven);
if (even) {
    console.log("all numbers are even");
} else {
    console.log("not all numbers are even");
}
```

4 测试

4.1 概述

结合前期的《软件需求规格说明书》和《软件工程总体设计报告》所确定的功能模块，以及测试本身所设计到的方面，拟将从如下角度对该软件做出详细的测试。在接下来的测试文档里面，会以各种功能模块进行测试，在模块里面，会涵盖表中所示的测试内容。

测试项目名称	测试目的	测试内容
内容测试(Content Testing)	检查呈现给用户语法、语义和组织结构，保证内容准确传达给用户	语法 语义 组织结构
数据库测试(Database Testing)	测试数据库能否正确运行	
用户接口测试(User Interface Testing)	测试用户能否通过网页界面完成想要执行的操作	主页面 详情页面 搜索页面 咨询页面 管理页面
导航测试(Navigation Testing)	测试书签能否正确显示、页面链接能否正确转跳	书签 页面链接
配置测试(Configuration Testing)	测试应用在服务端核客户端占用的资源 ¹	服务端 客户端
可用性测试(Usability	利用黑盒测试系统功能是否	登录

¹ 对于客户端，我们的web应用占用的资源主要取决于访问应用的浏览器

Tests)	齐全，各个功能是否正确执行	注销 查询谣言（普通用户） 查看谣言（普通用户） 发起咨询（普通用户） 发布澄清信息（管理员） 发布公告（管理员） 更改用户权限（管理员） 解答用户咨询（专家）
边界测试(Boundary Testing)	测试程序对边界情况是否正确处理	登录 注销 查询谣言（普通用户） 查看谣言（普通用户） 发起咨询（普通用户） 发布澄清信息（管理员） 发布公告（管理员） 更改用户权限（管理员） 解答用户咨询（专家）
兼容性测试(Compatibility Testing)	测试应用在不同操作系统、浏览器、屏幕分辨率下能否正常显示、运行	操作系统 浏览器 屏幕分辨率
性能测试(Performance Testing)	测试系统在高负载情况下的功能和性能的承受情况	登录 注销 查询谣言（普通用户） 查看谣言（普通用户） 发起咨询（普通用户） 发布澄清信息（管理员） 发布公告（管理员） 更改用户权限（管理员） 解答用户咨询（专家）

4.2 测试结果

4.2.1 内容测试

测试内容	测试结果
信息事实准确	通过
信息简洁明了	通过
内容布局易于用户理解	通过
用户可以轻松找到内容对象中的信息	通过
提供信息的正确引用或参考	通过
呈现的信息不自相矛盾	通过
呈现的信息与其他内容对象中呈现的信息不矛盾	通过
内容不令人反感	通过
内容没有误导性	通过
内容不侵犯现有的版权或商标	通过
内容包含补充现有内容的内部链接且链接正确	通过
内容的美学风格与界面的美学风格不冲突	通过

4.2.2 数据库测试

● 增删查

数据表	增	删	查
user	正常	正常	正常
information	正常	正常	正常
problem	正常	正常	正常
answer	正常	正常	正常

● 外键约束

数据表字段	外键功能
Information.uid	正常
problem.uid	正常
answer.qid	正常

4.2.3 用户接口测试

用户接口即对用户开放的接口信息，用户和服务器之间通过接口传递交互信息，从而实现对信息的处理和获取。其中我们设计的所有接口的返回信息均为IStatus格式，这是我们自定义的一个信息规范格式，用于表达信息获取的正确性和信息载体。

```
interface IStatus{
```



```
status: boolean; // 状态
data?: IObject; // 数据内容
detail: string; // 具体信息
}
```

测试内容	接口路径	测试结果
热搜词	/hotNews	通过
最新消息	/allNews	通过
最新提问	/hotQuestion	通过
最热提问	/newQuestion	通过
我的提问	/myQuestion	通过
发送提问	/operation/question/create	通过
搜索消息	/searchResult/:word	通过
登录	http://account.fishstar.xyz/	通过
回调	/token	通过
具体信息	/rumor/news/id	通过
个人信息	/account/user/uid	通过

4.2.4 导航测试

测试内容 ²	测试结果
首页 -> 搜索页	正常
首页 -> 提问页	正常
首页 -> 信息页	正常
信息页 -> 首页	正常
提问页 -> 首页	正常
搜索页 -> 首页	正常
信息页 -> 搜索页	正常
搜索页 -> 信息页	正常
提问页 -> 登录页	正常
登录页 -> 提问页	正常

4.2.5 配置测试

² A->B表示从页面A跳转到页面B

对于服务端，由于我们目前硬件资源有限，只有一台低配阿里云服务器（1核1G）可以进行部署，性能测试中表现出现有硬件资源非常有限，测试为应用分配更少硬件资源意义不大，因此暂不做服务端配置测试。

对于客户端，我们在华为荣耀V9（6G内存）上使用四款浏览器分别访问网站，并查看内存使用状况。其中，两款浏览器使用了缓存的进程（Via，Firefox），两款浏览器未使用了缓存的进程（夸克，UC）。四款浏览器占用同内存差异较大，这与浏览器内部实现和设置有较大关系，但整体看来，目前市面上的主流智能手机都可以支持我们应用（本质上，取决于是否支持浏览器）。

4.2.6 可用性测试

测试内容	测试结果
登录&注销	通过
搜索&查看	通过
提问	通过

4.2.7 边界测试

测试内容	测试结果
密码长度限制	通过
提问内容为空时无法按提交	通过
提交内容字数不得多于1000	通过
搜索内容为空时无法按下搜索按钮	通过

4.2.8 兼容性测试

对于兼容性测试，我们在移动端上选择了不同操作系统、不同浏览器、不同屏幕分辨率以及不同系统字体大小与显示大小³进行测试。测试内容以及结果如下：

测试内容	测试结果
页面布局一致	通过
内容显示无误	通过
用户能够正常操作	通过
应用功能正常	通过

³ 根据华为手机设置中自带的解释，“字体大小”仅控制字体的大小，“显示大小”控制文字、图片等界面元素的大小。

加载速度正常	通过
--------	----

4.2.9 性能测试

在项目部署在服务器后，我们对项目在高负载情况下的表现进行测试。这里采用了loadster进行压力测试，录制脚本产生虚拟用户重复访问网站，测试结果也暴露了一些我们项目的不足之处。

测试规模	测试结果
200人同时访问	并发人数接近200人开始出现错误，随后出现大量错误，主要为连接超时，连接重置和socket超时。
50人同时访问	并发用户在到达50一段时间后开始出现错误，主要错误类型为HTTP 502和socket超时。
25人同时访问	并发用户在到达25一段时间后开始出现错误，主要错误类型为HTTP 502和socket超时和连接重置。
1人重复访问	当一个人不断重复操作时，程序也会出现不稳定的情况，当次数过多时，会出现超时现象。

本项目目前对于高并发的需求不能满足，推断的原因是受限于服务器性能（一核1G），mysql连接池在多次连接释放后容易出现超时，连接重置现象。

4.3 总结分析

通过此次测试，我们看到项目的一些优点，更看到了存在的一些问题。主要存在的问题是项目对高并发的支持不够好，与我们的预期目标相差较远；另外项目的功能也存在不够完善的地方。我们会在接下来的开发过程中针对目前存在的问题进行改进。

我们对所有测试项目的分级评价（良、中、差）如下：

测试项目名称	测试结果
内容测试(Content Testing)	良
数据库测试(Database Testing)	良
用户接口测试(User Interface Testing)	良
导航测试(Navigation Testing)	良
配置测试(Configuration Testing)	良

可用性测试(Usability Tests)	中
边界测试(Boundary Testing)	良
兼容性测试(Compatibility Testing)	良
性能测试(Performance Testing)	差